A Batch-Aware Inventory Engine for Perishable Goods

MSCS 532: Algorithms and Data Structures

Amar Kulkarni

Harika Rangisetty

Namratha Kondi

Sanya Kamra

November 6, 2026

Dr. Solomon

Application Context

A grocery store often receives the same product in several deliveries, each with a different expiration date. If the store records only one total for milk, it may promise cartons that are no longer sellable. For example, Milk Batch B101 has 20 cartons expiring November 7, while B102 has 30 cartons expiring November 15. Before the first cutoff, 50 cartons can be sold (assuming none are reserved); once B101 expires, only B102's 30 cartons remain available. The expired cartons still physically exist but must be removed from sellable inventory.

Our proposed Python application receives batches, looks up stock, reserves and fulfills orders, releases canceled reservations, and processes expired lots. It is designed for frequent exact lookups and stock checks, with fewer receiving and checkout updates. We will test an assumed 90% read / 10% write workload; this is a benchmark scenario, not an observed retail statistic. Batch-level issuing and disposal are important in perishable inventory decisions (Haijema, 2014; Ketzenberg et al., 2018).

Data Structures and Why We Chose Them

1. Hash table – batches_by_id. A Python dictionary maps a unique batch ID to the authoritative Batch record: sku, expiry, on_hand, reserved, and availability_version. Batch-ID lookup, insertion, and deletion are O(1) on average. An unsorted list would require O(B) searching; a balanced search tree would add complexity when batch IDs do not need to be ordered.

2. Per-product min-heaps – lots_by_sku. A dictionary maps each SKU to a min-heap of (expiry, batch_id, availability_version). This lets us select the earliest usable batch of milk, yogurt, or another product under first-expire, first-out (FEFO). Inserting or popping a heap entry costs O(log H), while checking a valid root is O(1). A FIFO queue prioritizes arrival date, not expiry date; a sorted Python list costs O(B) to insert.

3. Store-wide min-heap – expiry_events. This heap contains (expiry, batch_id) entries for all products. It tells the store which batch expires next without checking every SKU's heap. The first

entry is available in O(1), and each removal costs O(log H). A single SKU heap cannot answer store-wide expiry questions efficiently.

Consistency Rules

The dictionary is the source of truth; heaps are indexes, not duplicate inventory records. Maintain $0 \le \text{reserved} \le \text{on_hand}$. For a valid, nonexpired batch, sellable quantity equals on_hand − reserved; after expiry it is zero. Reservations must be all-or-nothing. When stock availability changes, increment availability_version so old per-SKU heap entries can be skipped later ("lazy invalidation").

Empty or expired batches are removed from the dictionary, and any affected reservations are reported for reconciliation. For this prototype, expiry begins at the start of the recorded expiry date; the implementing group must use that rule consistently.

Core Operations and Expected Complexity

Let B = number of live batches, H = number of heap entries (including stale entries), m = batches used by one order, s = invalid entries cleared, and e = due expiration entries.

Operation	Expected cost	Action
Find a batch by ID	O(1) average	Dictionary lookup.
Receive a new batch	O(log H)	Add record and push into both heaps.
Find next usable lot for SKU	O((s + 1) log H)	Discard stale heap roots; inspect earliest valid lot.
Reserve quantity across m lots	O((m + s) log H)	Precheck supply; commit allocations only if sufficient.
Release / fulfill m lots	O(m log H)	Update quantities and versioned heap entries.
Process e expiration events	O(e log H) + reconciliation	Pop due global events; remove expired records.
Total available units for one SKU	O(B) initially	Sum nonexpired, unreserved units; cache later if needed.

Implementation Handoff and Trade-offs

Each batch represents one SKU and one expiry, even if one supplier shipment contains several products. The next group should implement receive(), next_usable_lot(), reserve(), release(), fulfill(), expire(), and a total-availability query. Use a Python dictionary for the actual records and heapq for the two indexes. Test duplicate IDs, insufficient stock, already-expired arrivals, cancellation, and two batches of the same SKU when just one expires. The system is single-process and in-memory; simultaneous writers and durable storage are out of scope. A balanced ordered index could support frequent arbitrary expiry-range queries in O(log B + r) time, but it adds index-maintenance work (Goh et al., 1996).

References

Goh, C. H., Lu, H., Ooi, B. C., & Tan, K.-L. (1996). Indexing temporal data using existing B+-trees. Data & Knowledge Engineering, 18(2), 147–165. https://doi.org/10.1016/0169-023X(95)00034-P

Haijema, R. (2014). Optimal ordering, issuance and disposal policies for inventory management of perishable products. International Journal of Production Economics, 157, 158–169. https://doi.org/10.1016/j.ijpe.2014.06.014

Ketzenberg, M., Gaukler, G., & Salin, V. (2018). Expiration dates and order quantities for perishables. European Journal of Operational Research, 266(2), 569–584. https://doi.org/10.1016/j.ejor.2017.10.005

AI Usage Appendix

Significant prompts included: “grocery inventory management”; “read heavy”; “perishable goods”; “what we pass in min heap and global min heap”; “if I have 2 batches of same product and 1 expires what about count”; and requests to compare alternatives and rewrite the Phase 1 brief.

The early AI explanation was too broad about “available” stock: selecting the next usable batch is not the same as counting every sellable unit for a SKU. We separated these operations and clarified that expiring B101 does not remove B102. An earlier complexity summary also treated release as O(m) without considering reinsertion into the heap; we changed it to O(m log H) when indexes are updated. The group should review these statements, confirm its actual AI use, and be ready to explain the design unaided.